

Assignment 2

Design and Analysis of Algorithms

Muhammad Masson /halid

211-5692

Question 1 :-

To sort the unsorted array A, we can make three pointers in the array low, mid and high pointers. The low and mid pointers will begin from the start of array and the high pointer will start from the end of array.

We will be checking the value of $arr[mid]$:-

- if red, swap $arr[low]$ and $arr[mid]$ also increment low and mid pointers.
- if white, increment mid pointer.
- if blue, swap $arr[mid]$ and $arr[high]$ also decrement high pointer.

Pseudocode :-

```
void sort(int arr[], int n) {
```

```
    int low = 0;
```

```
    int mid = 0;
```

```
    int high = n - 1;
```



```

while (mid <= high) {
    if (arr[mid] == red) {
        swap(arr[low] and arr[mid]);
        low++; mid++;
    }
    else if (arr[mid] == white) {
        mid++;
    }
    else {
        swap(arr[mid] and arr[high]);
        high--;
    }
}
}

```

Question 2:-

The key to pick is that both the arrays are sorted. We can make two pointers in array 1, we can make the last array element (the largest element) as the pointer and in the array 2 we can make the first element (the lowest element) the second pointer. Now, we will compare both the elements of the array pointers if the element of array 2 is smaller than that of array 1, we will swap them ^{and decrement pointer 1 and increment pointer 2} and we will do this until we don't find an element in array 2 that is larger than that in array 1, then we can simply sort the both arrays and merge them.

Pseudocode :-

```
void merge(int arr1[], int arr2[], int m, int n) {  
    int left = n - 1; pointer for 1st array the largest element  
    int right = 0; pointer for 2nd array the smallest element  
    while (left >= 0 && right < m) {  
        if (arr1[left] > arr2[right])  
        {  
            swap(arr1[left], arr2[right]);  
            left --, right ++;  
        }  
        else {  
            break;  
        }  
    }  
    sort(arr1, arr1 + n);  
    sort(arr2, arr2 + m);  
}
```

Question 3 :-

To get the count, we will check if the array is not rotated. If it is, return 0, as no rotations. We will make 2 pointers: start and end of array. We will calculate the middle index and check that $arr[mid]$ is less than, or equal to both its previous and next elements, if yes, return mid as the index of minimum element. If $arr[start]$ is less than or equal to

$arr[mid]$, the left part is sorted.
We will move start to $mid+1$ to search in right, unsorted part. If $arr[mid]$ is less than or equal to $arr[end]$, it means the right part is sorted. We will move end to $mid-1$ to search in the left, unsorted part. This algorithm will find the index of the minimum element which will be the number of the count of rotations.

Pseudocode :-

```
void sort findindex (int arr[]) {  
    int n = arr.size();  
    int start = 0, end = n-1;
```

```
    // If array is not rotated  
    if (arr[start] <= arr[end]) {  
        return 0;  
    }
```

```
    while (start <= end) {
```

```
        int mid = start + (end - start) / 2;
```

```
        int prev = (mid + n - 1) % n;  
        int next = (mid + 1) % n;
```

```
        if (arr[mid] <= arr[prev] && arr[mid] <= arr[next]) {  
            return mid;
```

```
        }
```



```

else if (arr[start] <= arr[mid]) {
    start = mid + 1;
}
else if (arr[mid] <= arr[end]) {
    end = mid - 1;
}
}

return -1;
}

```

Question 4:-

Q

If the array contains only one element, that is the majority element. If not, we split array in 2 halves, recursively finding the majority elements of each half. If both have same majority element, then this element is returned. If not, we will count the occurrences of each candidate majority element for the entire array. If candidate has count greater than half, it is returned, if neither, none is returned as no majority element exists.

```

void Majority (int arr[]) {
    n = len(arr)

```

```

// If array has only one element
if (n == 1)
    return arr[0];

```



```
mid = n // 2;  
A1 = arr[:mid];  
A2 = arr[mid:];
```

```
majority1 = find Majority(A1);  
majority2 = Majority(A2);
```

```
if majority1 == majority2:  
    return majority1
```

```
// count occurrence of each candidate  
// majority element
```

```
count1 = arr.count(majority1)  
count2 = arr.count(majority2)
```

```
if count1 > n // 2:  
    return majority1;  
elif count2 > n // 2:  
    return majority2;  
else:  
    return None
```

b

We will initialize two variables candidate and count, then we will iterate through the array, if count is 0, we will set the current element as the candidate and reset count to 1. If the current element is the same as the candidate, increment count by 1 if not decrement by 1. After iterating, if there is a

majority element, it will be stored in candidate. Check if the count of candidate in the array is greater than half the length of the array. If it is, return the candidate as the majority element else return None as there is no majority element.

PseudoCode

```
void Majority (int arr[]) {
```

```
    int candidate = 0;
```

```
    int count = 0;
```

```
    for (int num : arr) {
```

```
        if (count == 0) {
```

```
            candidate = num;
```

```
            count = 1;
```

```
        } else if (candidate == num)
```

```
        { count += 1; }
```

```
        else { count -= 1; }
```

```
    }
```

```
    int candidate_count = 0;
```

```
    for (int num : arr) {
```

```
        if (num == candidate)
```

```
        { candidate_count += 1; }
```

```
    }
```

```
}
```



```

    if (candidate_count > arr.size() / 2) {
        return candidate; // majority element
    } else {
        return -1; // no majority element
    }
}

```

Question 5 :-

a

```

int heap_count = 0;
int merge_count = 0;
int quick_count = 0;

```

```

void heapify (int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

```

```

    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

```

```

    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

```

```

    heap_count++;

```

```

    if (largest != i) {
        swap (arr[i], arr[largest]);
        heapify (arr, n, largest);
    }

```

```

}

```



```

void heapSort (int arr[]) {
    int n = arr.size();

```

```

    for (int i = n/2 - 1; i >= 0; i--) {
        heapify (arr, n, i);
    }

```

```

    for (int i = n - 1; i > 0; i--) {
        swap (arr[0], arr[i]);
        heapify (arr, i, 0);
    }
}

```

```

void merge (int arr[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

```

```

    int L[n1];
    int R[n2];

```

```

    for (int i = 0; i < n1; i++) {
        L[i] = arr[l + i];
    }

```

```

    for (int j = 0; j < n2; j++) {
        R[j] = arr[m + 1 + j];
    }

```

```

    int i = 0, j = 0, k = l;

```

```

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

```



```
k++;  
merge-count++;  
}
```

```
while (i < n1) {  
    arr[k] = L[i];  
    i++; k++; }  
}
```

```
while (j < n2) {  
    arr[k] = R[j];  
    j++; k++; }  
}
```

```
void mergeSort(int arr[], int l, int r) {  
    if (l < r) {  
        int m = (l + (r - 1)) / 2;  
        mergeSort(arr, l, m);  
        mergeSort(arr, m + 1, r);  
        merge(arr, l, m, r); } }  
}
```

```
int partition (int arr[], int low, int high) {  
    int pivot = arr[high];  
    int i = (low - 1);
```

```
    for (int j = low; j <= high - 1; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            swap(arr[i], arr[j]);  
        }  
    }
```

```
    quick-count++;  
}
```

```
swap(arr[i + 1], arr[high]);  
return (i + 1);  
}
```



```

void quickSort (int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort (arr, low, pi - 1);
        quickSort (arr, pi + 1, high);
    }
}

```

b

```

class MinHeap {
    int heap;
    int heap-size;

    int parent (int i) { return (i-1)/2; }
    int left (int i) { return 2*i+1; }
    int right (int i) { return 2*i+2; }

public:
    MinHeap();
    void insertKey (int key);
    int extractMin();
    void decreaseKey (int i, int new-val);
    void deleteKey (int i);
    int getMin();
};

```

```

MinHeap::MinHeap() {
    heap-size = 0;
}

```

```

void MinHeap::insertKey (int key) {
    heap-size++;
    int i = heap-size - 1;
    heap.push_back (key);
}

```



```

while (i != 0 && heap[parent(i)]
    > heap[i]) {
    swap(heap[i], heap[parent(i)]);
    i = parent(i);
}
}

```

```

int MinHeap::extractMin() {
    if (heap-size <= 0)
        return INT_MAX;

```

```

    if (heap-size == 1) {
        heap-size--;
        return heap[0];
    }

```

```

    int root = heap[0];
    heap[0] = heap[heap-size-1];
    heap-size--;

```

```

    int i = 0;
    while (i < heap-size) {
        int left-child = left(i);
        int right-child = right(i);
        int smallest = i;

```

```

        if (left-child < heap-size && heap[
            left-child] < heap[smallest]) {
            smallest = left-child;
        }

```

```

        if (right-child < heap-size && heap[
            right-child] < heap[smallest]) {
            smallest = right-child;
        }
    }
}

```



```

if (smallest != i) {
    swap(heap[i], heap[smallest]);
    i = smallest;
}
else
{
    break;
}
}
return root;
}

```

```

void MinHeap::decreaseKey(int i,
int new_val){
    heap[i] = new_val;

```

```

while (i != 0 && heap[parent(i)] >
heap[i]) {
    swap(heap[i], heap[parent(i)]);
    i = parent(i);
}
}

```

```

void MinHeap::deleteKey(int i)
{
    decreaseKey(i, INT_MIN);
    extractMin();
}

```

```

int MinHeap::getMin() {
    return heap[0];
}

```